

PYTHON



Python programming detail guide.

1. Script.py

⇒ python code file, it runs python programs.

⇒ python ka code kisi program chalan kar sakta hai.

2. data-analysis.ipynb

⇒ used to write and run python code for data analysis

⇒ data analysis ke liye python code likh kar chalan sakta hai.

3. program.exe

⇒ A program / application file that can run on window

⇒ window mein program / application chalan kar sakta hai.

4. document.docx

⇒ A microsoft word document for writing text.

⇒ letter, report, assignment aur kisi bhi karte hain likh kar chalan sakta hai.

5. Spreadsheet.xlsx

⇒ An excel file for storing and calculating data

⇒ Data rakhna aur kisi kisi kaable se chalan sakta hai.

6. Image.png

⇒ An image / picture file

⇒ photo ka picture rakhna chalan sakta hai.

7. archive.zip

• combines / compress files into one file.

• अलग-अलग files को एक साथ compress file में 2120d
खुला देते हैं।

8. styles.css

• controls the design and appearance of a website.

• website की colour size, font, layout
आदि निर्धारित करता है।

Short cuts

• .py → Python code

• .ipynb → data analysis / Jupyter notebook

• .exe → Program चलाने के लिए

• .docx → word document

• .xlsx → Excel / spreadsheet

• .png → image

• .zip → files compress

• .css → website design.

`print("Hello, world!")`

↓

`print()` 2301 function है।

"Hello, world" output है जो run के बाद
"Hello, world" आएगा.

* Python Indentation

indentation अर्थात् code के अगला? space
होना है। Python में यह ~~code~~ अर्थात्
अगर यह अर्थात् indentation अर्थात्

Example

python

if 5 > 2:

यहाँ print अर्थात् space है। यहाँ तो 20 को
अर्थात् code है।

example

* Python

if 5 > 2:

print("Five is greater than two!")

यहाँ print अर्थात् space है। यहाँ तो 20 को
अर्थात् code है।

print function

yo main chahi hamle double
quotes or () quotes hona pani milke

variable → variable name अर्थात् data
अर्थात् डिस्टिक्ट नाम है।

python

name = "john"

यहाँ name है "john" अर्थात्
अर्थात् data है।

*

Python Data Structure

`x = "Hello world"` $\Rightarrow$ string

`x = 20` $\Rightarrow$ int

`x = 20.5` $\Rightarrow$ float

`x = ["apple", "banana", "cheery"]` $\Rightarrow$ ~~complex~~ ^{list} ~~complex~~

`x = ("apple", "banana", "cheery")` $\Rightarrow$ tuple

`x = 1j` $\Rightarrow$ complex

`x = range(6)` $\Rightarrow$ range

`x = {"name": "John", "age": 36}` $\Rightarrow$ dict

`x = {"apple", "banana", "cheery"}` $\Rightarrow$ set

`x = frozenset({"apple", "banana", "cheery"})` $\Rightarrow$ frozenset

`x = True` $\Rightarrow$ boolean

str : text or string variable

int : whole number [number]

float : decimal number [point vako]

list : collection of item [data dastai tham mah rakh]

tuple : fixed collection [change garna nahi]

complex : complex number [jastaki number ra letter (2)]

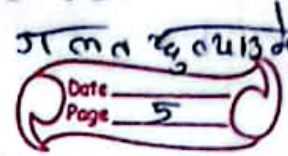
range : sequence of number

dict : key value data (key ra value mah data rakhe)

set : unique item [duplicate data]

Pro lensed! fixed / unchangeable set
Boolean: true or false (2-True 2-False)

variable name



variable name can have short name like
x and y

* Rules of python variable

- variable name must start with letter or the underscore character.
- variable name cannot start with number.
- variable name can only contain alpha numeric character, and underscore (A-Z, 0-9, and -)
- variable names case sensitive (age - Age and AGE are three variables)
- A variable name cannot be any of the python keywords.

* multiple string

aaba hamirai dherai string variable lekhaun
xavanera hamirai chahi 3 ta quotes ike "" "" "" use
garene garna sakxau.

* python string modify

1. `=> upper()` :-

upper vane ko hamirai upper use
garene sabai letter haru lai botto alphabete
mah convert garna sakxau eg: in here hello convert

HELLO

name = "hello"

print (name.upper()) output [HELLO]

2) lower() :-

Bahut letter ko small letter mai convert
garha eg:- HELLO convert hello

name = "HELLO"

3)

print(name.lower())

output: hello

3) strip() :-

String ko start ra end ko extra space

hatawa ya space hatawa

name = " — Hello — "

print(name.strip())

output: Hello

4) replace string

4) replace :-

replace ko koi a letter or word ko
dusra letter or word mai convert garha

eg:- a = "Hello, world!"

print(a.replace("H", "J"))

"H" = old letter

"J" = new letter

chota H ko thoda mai J hunta

output: Jello, world!

* Boolean value

* Boolean means there are only two possible
answer that mean true or false.

eg:- create a list
thisList = ["apple", "banana", "cherry"]
print(thisList).

* ordered

ordered जे. vaneko item हिन अन क्रम
order मा 2112923को xa vane छोटी क्रम
कायन रहेको हो।

* changable

list changable हुन्छ, किनकि list
Banayeko अन item change, add &
remove गर्न सकिन्छ।

eg:-

```
fruits = ["apple", "banana", "cherry"]
fruit[0] = "mango"
print(fruits)
```

output:-

```
["mango", "banana", "cherry"]
```

* Negative indexing

Negative indexing means
start from the end.

* append item

append le list ko last/end मा naya
item add गर्छ।

* Insert Item

insert() ka list ki aayus chayeko index position ma naya item thapxa.

* Extend List

extend() le auna list ko sabai item auni list ma thapxa.

extend() is used to add all items from one list to another list.

* Remove Specified Item

remove() is used to remove specific item from a list.

eg:-

```
this list = ["a", "b", "c"]
```

```
this list.remove("b")
```

```
print(this list)
```

* Remove Specified Item Index

pop() le list bata specific index / position ki item hatauxa

eg:-

```
this list = ["a", "b", "c"]
```

```
this list.pop(1)
```

```
print(this list)
```

print garxa 1 ko position

hatauxa.

* clear the list

clear le list ko vira sabai item hatayxa, list ma vakeko

eg.

Jasti list ma dherai item haruxa tara hamile clear() print garyau vane sabai hatxa. ani ~~[]~~ kai pani print hudaina

* for loop

for loop le list vira garkai ek ek item lai one to one access/print garna madad garxa.

eg:

this list = ["a", "b", "c"]

for x in this list:

print(x) → aaba char

x ni one to one garera list ki item lai %

ani this list ki item one to one garera x ni print %

ani

* for loop le list ko item one by one access/ print %

* loop through the index number

list ko items lai index number use sth one by one access garna sakinx python ma index 0 bata suru %

* Sort list alphabetically
sort() le list ko item lai alphabetical
numerical me sort karta hai.

Jaghai

```
num = ["2", "4", "6", "1", "3", "5"]
```

```
num.sort()
```

```
print(num)
```

```
["1", "2", "3", "4", "5", "6"] aise aayega
```

* list method

list() method is used to create or convert
another collection into a list.

eg:-

```
x = list("a", "b", "c")
```

```
print(x)
```

output

```
["a", "b", "c"]
```

* Join two list

we can join two lists using the + operator

* Python has a set a built-in method that you can use a list.

`append()` :- Add an element at the end of the list

`clear()` :- remove all the elements from the list

`copy()` :- Return copy of the list

`count()` :- Return the number of the element with the specified value

`extend()` :- Add the element of a list (or any iterable) to the end of the current list

`index` :- Return the index of the first

`insert` :- Add an elements at the specified positions.

`pop()` :- Remove the elements at the specified position

`remove()` :- Remove the item with the specified value.

`reverse()` :- Reverses the order of the list

`sort()` :- sorts the list

* tuple

A tuple is used to store multiple items in a single variable.



Important points

1. ordered

It's have a fix order.

2. unchangeable

once a tuple is created, its items cannot be changed, added, or removed.

3. round brackets ()

tuple banana round brackets ()

use ~~han~~ garinxa.

eg:-

```
fruit = ("apple", "Banana", "cheery")
```

```
print(fruit)
```

output:-

```
("apple", "banana", "cheery")
```

* tuple item

A tuple item is each individual value stored inside a tuple.

easy way

* tuple : Ohera items ko collection

tuple item : tuple vatra rasta value .

* Allow duplicate

Allow duplicate means the same value can appear more than once.

eg:-

```
Fruits = ("apple", "Banana", "cherry", "apple")
```

output

```
("apple", "Banana", "cherry", "apple")
```

Tuple can allow duplicate item.

* Tuple length

len() is used to find the number of item in a tuple.

* Tuple Item-Data type

Tuple items can have different data type such as string, integer, float and boolean.

eg:-

```
my tuple = ("navrav", 26, 5.5, True)
```

```
print(my tuple)
```

navrav — String

26 — Integer

5.5 — Float

True — Boolean

```
output ("navrav", "26", "5.5", True)
```

* Access tuple Item

Tuple ko item access karne ke liye index number use karin. Python mein index 0 se start hoga.

* Dictionary

A dictionary stores data in key-value pairs and a dictionary is a collection which is ordered, changeable and do not allow duplicates.

key - Data name, Identity
value - actual data.

* Change values

You can change the value of a specific item by referring to its name.

* Adding Item

We can add a new item to a dictionary by using a new key.

eg:-

```
student = {
```

```
    "name": "Naurav",
```

```
    "age": 26
```

```
}
```

```
student["department"] = "IT"
```

```
print student
```

output :-

```
{'name': 'Naurav', 'age': 26, 'department': 'IT'}
```

pop() → key by key
remove

Date _____
Page 16

* Removing Item

There are several method to remove item from a dictionary.

pop() remove an item from a dictionary using the key name

eg:-

```
thisdict = {  
    "b": "Ford"  
    "a": "Model"  
    "y": "2008"  
}
```

```
thisdict.pop("Model")  
print(thisdict)
```

now model remove than output is
{~~"a": "Ford"~~, "y": "2008"}

copy()

copy() it of original dictionary creates copy Banuxa.

* Different Between tuple and list

tuple	list
- tuple is a collection that is immutable	- list is a collection of elements that is mutable
- we cannot change, add or remove element.	- we can change, add or remove element
- tuple is written using parentheses <code>()</code> .	- list is written using square bracket <code>[]</code>

* Dictionary method.

- copy() ⇒ Dictionary ko copy banana.
- fromkeys() ⇒ Pykeko key bata naya key banne.
- get () ⇒ key ko value nikalna
- items() ⇒ sabai key value pairs dinna
- keys() ⇒ sabai key dinna
- pop() ⇒ Pykeko key item hatauna
- popitem() ⇒ Remove the last inserted item
- set default() ⇒ key navayeko naya key/value add garxa.
- update() ⇒ Dictionary lai update / add garxa.
- value() ⇒ 2-4 value likhna

copy() ⇒ copy banana

get() ⇒ value ko nikalne

key() ⇒ keys nikalne

values() ⇒ values nikalne

item() ⇒ key + value

pop() ⇒ item remove

popitem() ⇒ last item hataune

update() ⇒ update / add

Python - L19
Date: 20/12/2020
Page: 3000

Random module.



* Python conditions and IF statement
Python supports the usual logical condition from mathematics

- Equals: $a == b$
- Not Equals: $a != b$
- less than: $a < b$
- less than or equal to: $a <= b$
- greater than: $a > b$
- greater than or equal to: $a >= b$

This condition can be used in several way most commonly in "if" statements and loops.

* multiple statement in if block
You can write multiple statement inside if block. All statements must be have the same indentation

eg:-

```
age = 20
```

```
if age >= 18:
```

```
    print("You are an adult")  
    print("You can vote")  
    print("You are eligible")
```

* using boolean variable in condition

A boolean variable contains true or false. We can use it directly in an if statement.

* The elif keyword

elif means "else if". It is used to check another condition when the first if condition is false.

eg:-

if condition # first condition

elif condition # second

else " # none of the condition

eg:-

b = 33

if b == a

print("b is greater than a")

elif a == b:

print("a and b are equal")

out put

a and b are equal

eg:- else

else runs when the if and elif condition are false

b = 33

if b > a: (false)

print("b is greater than a")

elif a == b: (false)

print("a and b are equal")

else: (true)

print("a is greater than b")

* ELSE without elif

You can use else directly after if, without using elif.

* Python logical operators

1. and

Returns true only when both condition are true.

2. or

Returns true when at least one condition is true.

3. not

Reverse the result: true $\rightarrow$ false | false $\rightarrow$ true.

Easy way

• and $\rightarrow$ दुबाई सँटिछे

("or" $\rightarrow$ आबुता वाये पुग्छा

not $\rightarrow$ उमेरि आउछे

* The pass statement

pass means do nothing. It is used when you need a statement but you don't want to write any code yet.

* Nested IF Statement

A NESTED IF IS AN STATEMENT INSIDE ANOTHER IF STATEMENT.

```
1 age = 20
```

```
2 has_id = True
```

```
3
```

```
4 if age >= 18: - first condition
```

```
5     if has_id: - second condition
```

```
6         print("you can enter")
```

```
7
```

```
8     output: you can enter - True
```

easy way

NESTED IF = IF INSIDE IF

```
1 name = "John"
```

```
2 age = 20
```

```
3 if name == "John":
```

```
4     if age >= 18:
```

```
5         print("John is an adult")
```

```
6
```

```
7
```

```
8
```

```
9
```

```
10
```

```
11
```

```
12
```

```
13
```

```
14
```

```
15
```

```
16
```

```
17
```

```
18
```

```
19
```

BMI PROGRAM

Date _____
Page 22

```
print("welcome to treasure island")  
print("your mission is to find the treasure")
```

```
choice1 = input("left or right").lower()
```

```
if choice1 == "left":
```

```
    choice2 = input("swim or wait?").lower()
```

```
if choice1 == "left":
```

```
    choice2 = input("swim or wait?").lower()
```

```
if choice2 == "wait":
```

```
    choice3 = input("which door? red, blue or  
yellow?").lower()
```

```
if choice3 == "yellow":
```

```
    print("you win!")
```

```
elif choice3 == "red":
```

```
    print("game over.")
```

```
elif choice3 == "blue":
```

```
    print("game over.")
```

```
else:
```

```
    print("invalid choice.")
```

```
else:
```

```
    print("game over.")
```

```
else:
```

```
    print("game over.")
```

Rock x Paper x Scissors

Date _____
Page 23

import random

print("Rock Paper Scissors game")

print("1. Rock")

print("2. Paper")

print("3. Scissors")

user = int(input("Enter your choice (1-3):"))

computer = random.randint(1, 3)

if user == 1:

 user_choice = "Rock"

elif user == 2:

 user_choice = "Paper"

elif user == 3:

 user_choice = "Scissors"

else

 print("Invalid choice")

 exit()

if computer == 1:

 computer_choice = "Rock"

elif computer == 2:

 computer_choice = "Paper"

elif computer == 3:

 computer_choice = "Scissors"

print("Your choice:", user_choice)

print("Computer choice:", computer_choice)

if user == computer:

```
print("Draw!")
```

```
elif user == 1 and computer == 3:  
    print("you win!")
```

```
elif user == 2 and computer == 1:  
    print("you win!")
```

```
elif user == 3 and computer == 2:  
    print("you win!")
```

```
else:
```

```
    print("computer win!")
```

* python loops

python has two primitive loop commands,

- while loops
 - for loops
- while loops repeats a block of code as long as the condition is true.

eg:-

```
i = 2  
while i <= 10:  
    print(i)  
    i = i + 2
```

output

2

4

6

8

* The break statement .

break is used to stop the loop immediately when a condition is met .

eg:-

i = 1

while i < 5:

print(i)

if i == 3:

break — break means loop stop.

i + 1

output

1

2

3

§

* continue statement

continue skips the current iteration and continues with the next iteration.

eg:-

i = 0

while i < 5:

if i == 3:

continue

print(i)

output

1

2

4

5

break → all loop stop

continue → turns skip and another turn continue.

* The break statement
Break is used to stop a loop immediately.

eg:-

```
i = 1
while i <= 5:
    print(i)
    if i == 3:
        break
    i += 1
```

output

1
2
3

when $i=3$, break was executed, so 4, and 5 were not printed.

* The range() function.
The range() function is used to generate a sequence of numbers, usually with a for loop.

Syntax:

range(start, stop, step)

eg:-

```
for i in range(5):
    print(i)
```

output

0
1
2
3

4

* python function

A function in python is a block of code that performs a specific task, you can define it once and use it many times.

Basic syntax

```
def function_name():
```

```
    # code
```

eg:-

```
def greet():
```

```
    print("Hello, world")
```

```
greet()
```

Output

Hello, world!

def → used to create a function

greet → function name

() → parameters go here

greet → calls / runs the function.

* Creating a function in python.

To create a function in python we use the keyword.

Syntax

```
def function_name():
```

```
    # code to execute
```

```
def greet():
```

```
    print("Hello, pratiksha!")
```

```
greet()
```

Output

Hello, pratiksha!

* calling a function in python

calling a function means running the code inside a function.

After creating a function we call it by writing the function name followed by ().

eg:-

```
def my-function():
    print("Hello, world!")
```

```
my-function()
```

output.

Hello, world!

* python return vs print()

The main difference is:

print - display something on the screen.

return - sends a value back from a function.

eg:-

1. using print()

```
def add(a,b):
    print(a+b)
```

```
result = add(10,20)
```

```
print(result)
```

output

30

None → Because the function print 30, but it does not return a value.

2. using return

```
def add(a,b):
```

```
    return a+b
```

```
result = add(10,20)
```

output :

30

Here return sends 30 back to result.

print()	return
- Shows output onscreen	- send value back
- mainly for displaying	- mainly for using the result
- doesnot give the value back to the caller	inves the value back
- Can be used outside function too	normally used inside function

* Calculator program.

```
import random
```

```
def love - calculator (name1, name2):  
    love = random.randint(1, 100)
```

```
    print ("\n. love calculator")
```

```
    print (name1, name2)
```

```
    print ("love percent:", love "%")
```

```
    if love >= 80:
```

```
        print ("wow perfect match!")
```

```
    elif love >= 60:
```

```
        print ("great match!")
```

```
    elif love >= 40:
```

```
        print ("good match")
```

```
    else:
```

```
        print ("keep trying!")
```

```
name1 = input("Enter first person's name :")
name2 = input("Enter second person's name :")
```

```
love_calculator(name1, name2)
```

Output

Am love sita

love percentage : 87%

wow ! perfect match.

* oop (object oriented program)

oop stand for object oriented programming

oop is a program method where a program is designed using classes and object.

eg:-

```
class student:
    name = "pratiksha"
    age = 23
```

```
student1 = student()
print(student1.name)
print(student1.age)
```

Output

pratiksha

23

student → class

student1 → object

name, age → Attributes

- * Advantage of pop.

 1. clear structure
 2. easy to maintain, reuse and debug
 3. reusable application.

* what are class and object.
class:-

A class is a blueprint or template for creating objects.

eg:-

```
student = Student()
print(student.name)
print(student.age)
```

object

An object is an instance of a class

eg:-

```
class student:
    name = "Ram"
student1 = student()
student2 = student()
```

* create a class

An object is a thing created from a class.

eg:-

```
class student:
    name = "pratiksha"
    age = 20
student1 = student()
```

* multiple object in python.
multiple objects means creating more than one object from the same class.

eg:-

```
class student:
    name = "pratiksha"
    age = 20
```

```
student1 = student()
```

```
student2 = student()
```

```
student3 = student()
```

```
print(student1.name)
```

```
print(student2.name)
```

```
print(student3.name)
```

output

```
pratiksha
```

```
"
```

```
"
```

The `__init__()` method

```
class student:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
student1 = student("pratik", 20)
```

```
student2 = student("Ram", 21)
```

```
student3 = student("Sita", 29)
```

```
print(student1.name)
```

```
print(student2.name)
```

```
print(student3.name)
```

output

pratik

Ram

Sita

* why do we use `__init__()`?

→ `__init__()` is used to set the initial values of an object when the object is created.

Think of it like a setup function that runs automatically when you create an object.

example without `__init__()`

```
class student:
```

```
    name = "Sita"
```

```
    age = 25
```

Every object gets the same values unless we change them manually.

example with `__init__()`

```
class student:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
student1 = student("Ram", 20)
```

```
student2 = student("Sita", 21)
```

```
print(student1.name)
```

```
print(student2.name)
```

output

Ram

* Default value in `__init__` ()

A 'default value' is a value that python uses automatically if you do not provide a value when creating the object.

eg:-

```
class student:
    def __init__(self, name, age=20):
        self.name = name
        self.age = age
```

```
student1 = student("pratik")
student2 = student("Naurav", 28)
```

```
print(student1.name, student1.age)
print(student2.name, student2.age)
```

Output

pratik 20

Naurav 28

How it's work

age 20 - means 20 is default value.

So

```
student("pratik")
```

No age given, so python use 20

But

```
student("Naurav", 28)
```

age is given, so python use 28 instead 20.

* multiple parameters in `__init__()`

multiple parameters means giving more than one parameter to `__init__()`.

eg:-

```
class student:
```

```
def __init__(self, name, age, department):
```

```
    self.name = name
```

```
    self.age = age
```

```
    self.department = department
```

```
student = student("pratik", 20, "computer")
```

```
print(student.name)
```

```
print(student.age)
```

```
print(student.department)
```

output

pratik.

20

computer

Here are `__init__` has 3 parameter

* The self parameter in python.

self. refers to the current object.

It is used inside a class to access the object attributes and methods.

* ~~access properties.~~

* class methods in python.

A method is a function that belongs to a ~~class~~ class. methods are used to define what an object can do.

eg:-

```
class person
```

```
def __init__(self, name):
```

```
    self.name = name
```

```
def greet(self):
```

```
    print("Hello, my name is " + self.name)
```

```
p1 = person("Emil")
```

```
p1.greet()
```

output

Hello, my name is Emil

* Types of inheritance in python.

1. single inheritance.

single inheritance means one child class inherits one parent class.

2. multiple inheritance

multiple inheritance means one child inherits from two or more parent classes.

3. multiple level inheritance.

multiple inheritance means inheritance happens through multiple levels, like
grandparent → parent → child.

eg:-

class grandparent:

def house(self):

print("grandparent's house")

class parent(grandparent):

def car(self):

print("parent's car")

class child(parent):

def bike(self):

print("child, bike")

Structure.

grandparent

↓

parent

↓

child

4. Hierarchical Inheritance.

Hierarchical inheritance means multiple child classes inherit from one parent class.

5. Hybrid Inheritance.

Hybrid inheritance is a combination of two or more types of inheritance.

* Polymorphism

polymorphism means "many forms".
method function of farak farak object me
farak kam garta hai isko polymorphism

* Types of polymorphism in python

1. Method overriding (dynamic polymorphism)

Method overriding happens when a child class provides its own version of a method that already exists in the parent class.

eg:-

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
```

```
class Dog(Animal):
    def sound(self):
        print("dog barks")
```

```
class Cat(Animal):
    def sound(self):
        print("cat meows")
```

```
dog = Dog()
```

```
cat = Cat()
```

```
dog.sound()
```

```
cat.sound()
```

output

dog barks

cat meows

2. Duck typing

Duck typing means python cares about what an object can do, rather than what type/class the object belongs to

eg:-

```
class phone:
    def call(self):
        print("calling from phone")
```

```
class computer:
    def call(self):
        print("calling from computer")
```

```
def make_call(device):
    device.call()
```

```
phone = phone()
computer = computer()
```

```
make_call(phone)
```

```
make_call(computer)
```

output

calling from phone

calling from computer.

* Super keyword in python.

The super() function is used in a child class to access method or properties of the parent class.

* Encapsulation

Encapsulation means wrapping data and method together inside a class.

data 2 method ko ek class ke andar rakha hai encapsulation ho.

eg:-

```
class Student
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(self.name)
        print(self.age)
Student1 = Student("Ram", 24)
Student1.display()
name, age, display inside a class.
```